

Практическая работа 2

Управление конфигурацией и сопровождение программного обеспечения

Конфликт изменений, отмена ошибки и выпуск версии

Продолжение основ Git. Вы создадите две независимые правки настройки, разрешите конфликт, отмените незаписанную и сохранённую ошибки разными способами, подготовите выпуск и проверите восстановление истории.

Формат: сначала смотрите слайды и слушаете объяснение, затем работаете самостоятельно. На первой паре — 55 минут практики, на второй — 60. Все команды вводятся в Git Bash, по одной строке, без символа \$. Регистрация и удалённый сервер не нужны.

Учебная задача

Проект содержит config.txt с одной строкой interval=60 и README.txt с описанием. interval — интервал проверки в секундах. Настоящий сервис не запускается. Ветка feature-fast предложит 30, ветка main — 120. По условию согласован итог 60: проверка не реже раза в минуту, более частые проверки пока не требуются.

Новое рабочее место

Используйте 04_Стартовый_проект_Git2_Windows.zip. Распакуйте его отдельно от предыдущей работы. Старый git-practice не нужен для запуска и сохраняется у вас. Не распаковывайте новую копию поверх уже выполненной попытки.

Итог сдачи

Фамилия_Git2.zip: work.bundle, release.zip, history.txt, report.txt. Сохраните архив и рабочие папки у себя. Загрузка в ЭИОС будет позднее, после объявления ссылки и срока. Исполняемые файлы Git и вся папка проекта в сдачу не входят.

Проверка. Читайте каждый шаг целиком до выполнения. Не повторяйте весь маршрут поверх готовой работы. При ошибке остановитесь и проверьте состояние.

A0. Windows и Git Bash

Если Git for Windows уже установлен после первой работы, повторная установка не нужна. Если его нет: откройте <https://git-scm.com/install/windows>, выберите установщик под архитектуру своего компьютера и завершите установку. Тип системы: Пуск → Параметры → Система → О системе. Используем Windows 10/11 и Git Bash.

1. В Проводнике найдите стартовый ZIP → правая кнопка → «Извлечь всё». Выберите папку в своих Документах. Откройте распакованную git2-start. В ней должны быть service-config, два помощника .cmd и шаблон отчёта.
2. Дважды щёлкните 00_CHECK_GIT.cmd. Должна появиться версия Git. Закройте окно проверки и запустите 01_OPEN_GIT_BASH.cmd. Оно откроет Git Bash внутри service-config.
3. Если помощник не нашёл установленный Git, откройте service-config в Проводнике → правая кнопка по свободному месту → «Open Git Bash here». В Windows 11 может понадобиться «Показать дополнительные параметры».

Теперь выполняйте в Git Bash

```
git --version  
pwd  
ls
```

Проверка. pwd заканчивается на /service-config. ls показывает README.txt и config.txt. Сам символ \$ приглашения не вводим.

Каждую команду вводите отдельной строкой и нажимайте Enter. Можно копировать из PDF; для вставки используйте правую кнопку или Shift+Insert. Не копируйте заголовки шагов и ожидаемый вывод. Если после незакрытой кавычки появился знак >, нажмите Ctrl+C и введите строку заново.

В новом starter ещё нет репозитория. Если вы уже проходили несколько шагов в этой папке, сначала посмотрите git status и историю. Не удаляйте .git ради повторной попытки.

A1. Исходная версия

В Git Bash, папка service-config

```
git init -b main
git config user.name "Ivan Ivanov"
git config user.email "student@example.invalid"
git add README.txt config.txt
git commit -m "Add service configuration"
git tag v0.1.0
```

В user.name замените Ivan Ivanov своим именем. Адрес student@example.invalid — допустимый учебный пример, это не регистрация. Обе настройки локальные и выполняются после git init. Git for Windows должен поддерживать switch и init -b; при ошибке неизвестного параметра обновите его с официальной страницы.

Проверка после команд

```
cat config.txt
git status
git --no-pager log --oneline
```

Проверка. config.txt: interval=60. Ветка main, working tree clean. Один коммит Add service configuration, тег v0.1.0.

README.txt описывает условный проект. Сейчас отслеживаются только README.txt и config.txt. Помощники и шаблон отчёта лежат снаружи репозитория, поэтому в коммит не входят.

Если коммит не создан из-за имени автора, исправьте настройки и повторите только команду commit. Если тег уже существует, проверьте, выполнен ли этот шаг раньше; не используйте принудительную замену тега.

A2–A3. Две линии изменений

A2. Создайте предложение feature-fast

```
git switch -c feature-fast
echo "interval=30" > config.txt
git diff
git add config.txt
git commit -m "Set interval to 30 seconds"
```

Проверка. Текущая ветка feature-fast. Файл содержит interval=30; создан коммит Set interval to 30 seconds. git status — чистое дерево.

Знак > полностью заменяет config.txt. Здесь это намеренно: файл состоит из одной строки. git diff показывает замену 60 на 30.

A3. Создайте независимое предложение в main

```
git switch main
echo "interval=120" > config.txt
git add config.txt
git commit -m "Set interval to 120 seconds"
git status
```

Проверка. Текущая ветка main. Файл содержит interval=120; коммит Set interval to 120 seconds сохранён. Рабочее дерево чистое.

Сначала переключитесь в main, затем записывайте 120. Если обе правки попали в одну ветку, запланированный конфликт не возникнет. Не меняйте порядок.

А4. Ожидаемый конфликт

Запускайте из `main` с чистым деревом

```
git merge feature-fast  
git status  
cat config.txt
```

Проверка. Ожидается `CONFLICT`, незавершённое слияние и `both modified` для `config.txt`. Это контрольная точка упражнения.

Типичный фрагмент файла — не команды

```
<<<<<<< HEAD  
interval=120  
=====  
interval=30  
>>>>>> feature-fast
```

Верхняя часть относится к текущей `main`, нижняя — к `feature-fast`. Настройки отображения конфликта могут добавлять информацию об общей версии. В любом случае маркеры не должны остаться в окончательном файле.

Git сохранил коммиты обеих веток и ждёт согласованного содержимого. Не выполняйте новый `init`, не создавайте тег и не переключайте ветки до завершения слияния.

Если конфликта нет

Fast-forward или Already up to date означает другую историю. Выполните `git --no-pager log --oneline --graph --all` и проверьте, есть ли отдельные коммиты со значениями 30 и 120 в разных ветках. Обратитесь к преподавателю; не рисуйте маркеры вручную.

A5–A7. Разрешение и проверка

A5. Запишите согласованное решение

```
echo "interval=60" > config.txt
cat config.txt
git diff --check
```

Проверка. В config.txt одна строка interval=60. git diff --check не выводит замечаний. Он проверяет текстовые признаки, а соответствие интервала условию проверяете вы.

A6. Подтвердите решение и завершите merge

```
git add config.txt
git commit -m "Merge interval settings: use 60 seconds"
git status
```

Проверка. Ветка main; working tree clean; незавершённого слияния нет.

A7. Проверьте структуру истории

```
git --no-pager log --oneline --graph --all
git rev-list --parents -n 1 HEAD
```

В точном маршруте сейчас 4 коммита. rev-list выводит 3 хеша: merge-коммит и двух родителей. Хеши индивидуальны и не обязаны совпадать с образцом. Запишите хеш merge-коммита и объяснение конфликта в черновик отчёта.

Если status пишет, что конфликты исправлены, но слияние продолжается, выполните пропущенный commit. Если остаётся unmerged, вернитесь к проверке файла и git add config.txt. Случайные команды сброса не требуются.

Б1–Б2. Два вида ошибки

Перед началом

Завершите часть А. `git status` должен показывать `main` и чистое дерево; `cat config.txt` — `interval=60`. Внутри незавершённого `merge` эту часть не начинайте.

Б1. Отмените известную незаписанную правку

```
echo "interval=0" > config.txt
git diff
git restore -- config.txt
cat config.txt
```

Проверка. Снова `interval=60`. `git status` — чистое дерево. Нового коммита нет. В этом блоке намеренно не выполняются `add` и `commit` между `echo` и `restore`.

`restore` берёт файл из индекса; в данном состоянии индекс совпадает с последним коммитом. Незаписанная правка будет отброшена. Команда применима здесь, потому что ноль заведомо учебная ошибка, от которой решили отказаться.

Б2. Теперь сохраните ошибку коммитом

```
echo "interval=0" > config.txt
git add config.txt
git commit -m "Set invalid interval"
git --no-pager log -1 --oneline
cat config.txt
```

Проверка. Последний коммит `Set invalid interval`; `config.txt` содержит `interval=0`. Запишите хеш ошибочного коммита. До Б3 других коммитов не создавайте.

Б3–Б4. Исправление и описание версии

Б3. Отмените ошибочный обычный коммит

```
git --no-pager log -1 --oneline
git revert --no-edit HEAD
cat config.txt
git --no-pager log -2 --oneline
```

Проверка. До revert последняя запись — Set invalid interval. После: interval=60; в истории отдельный Revert и исходная ошибка. Рабочее дерево чистое, всего 6 коммитов.

HEAD здесь означает последний ошибочный коммит. Если после него вы сделали другой коммит, сначала уточните нужный хеш с преподавателем. Не отменяйте этим шагом merge-коммит и не повторяйте revert без проверки.

Б4. Создайте CHANGELOG.txt и сохраните

```
echo "v0.2.0" > CHANGELOG.txt
echo "Resolved interval conflict: 60 seconds." >> CHANGELOG.txt
echo "Reverted invalid zero interval." >> CHANGELOG.txt
git add CHANGELOG.txt
git commit -m "Document v0.2.0 release"
```

> заменяет текст файла; >> дописывает строку. Проверьте cat CHANGELOG.txt: номер версии и две строки пояснений. Отчёт заполняется по-русски; текст CHANGELOG дан латиницей для простого воспроизводимого ввода.

Проверка. Теперь 7 коммитов. CHANGELOG.txt включён в историю, config.txt по-прежнему содержит interval=60.

Б5–Б6. Выпуск и файлы передачи

Б5. Отметьте проверенную версию

```
git tag -a v0.2.0 -m "Training release v0.2.0"
git status
git tag --list
git rev-list --count --all
```

Проверка. Два тега: v0.1.0 и v0.2.0. Рабочее дерево чистое. Для точного маршрута 7 коммитов. Дополнительные осмысленные исправления объясните в отчёте.

Аннотированный v0.2.0 ставится после коммита CHANGELOG. Если тег уже существует, сначала проверьте его: `git show --stat v0.2.0`. Не переносите его принудительно ради повторного выполнения шага.

Б6. Создайте архив файлов и переносимую историю

```
mkdir -p ../submission
git archive -o ../submission/release.zip v0.2.0
git bundle create ../submission/work.bundle --all
git bundle verify ../submission/work.bundle
```

Проверка. В соседней папке submission появились release.zip и work.bundle. verify сообщает is okay. release.zip содержит README.txt, config.txt и CHANGELOG.txt.

release.zip не содержит истории Git. work.bundle содержит сохранённую историю и ссылки нашего репозитория, но не незакоммиченные файлы. Путь ../submission расположен рядом с service-config.

Б7. Восстановление и текст истории

Проверьте bundle в новой папке

```
git clone ../submission/work.bundle ../restore-check
git -C ../restore-check status
cat ../restore-check/config.txt
git -C ../restore-check tag --list
```

Проверка. restore-check создана рядом с service-config. В копии interval=60, теги v0.1.0 и v0.2.0, чистое рабочее дерево. Интернет не используется.

Параметр -C обращается к другой папке без смены текущей. После этих команд вы по-прежнему в service-config. Не выполняйте cd в restore-check для следующих команд.

Если restore-check уже существует, сначала проверьте результат прежнего клонирования. Для новой проверки можно выбрать новое имя restore-check-2, заменив его во всех четырёх командах. Не удаляйте рабочие каталоги вслепую.

Сохраните историю и подготовьте отчёт

```
git --no-pager log --oneline --graph --all --decorate > ../submission/history.txt
cp ../report_template.txt ../submission/report.txt
```

Первую длинную команду вводите одной строкой. При копировании переноса из PDF убедитесь, что команда не разорвана. Копирование шаблона ср выполняйте только если report.txt ещё не заполнен: повторная команда заменит ваш отчёт шаблоном.

Проверка. Теперь в submission четыре файла. Откройте report.txt через Проводник → «Открыть с помощью» → Блокнот и заполните все поля.

Б8. Архив для последующей сдачи

1. В report.txt заполните имя, ветки и значения, причину конфликта, согласованное решение, хеши слияния, ошибки и отмены, число коммитов и теги, результат проверки bundle и клонирования. Объясните разницу restore / revert и release.zip / work.bundle.
2. Нажмите Ctrl+S в Блокноте. В Проводнике включите расширения имён файлов: Windows 10 → Вид; Windows 11 → Вид → Показать. Проверьте, что имена не получили лишнее .txt.
3. Откройте release.zip. Внутри должны быть README.txt, config.txt и CHANGELOG.txt. Конфигурация содержит interval=60. Закройте просмотр архива.
4. Выделите work.bundle, release.zip, history.txt и report.txt в submission. Windows 11: «Сжать в ZIP-файл». Windows 10: «Отправить» → «Сжатая ZIP-папка». Назовите внешний архив Фамилия_Git2.zip. Откройте и проверьте все четыре вложения.
5. Сохраните архив, service-config и restore-check до проверки работы. Загрузите итоговый ZIP в ЭИОС после объявления ссылки и срока; подтвердите отправку, если система показывает такую кнопку. Во время занятия загрузка не требуется.

Оценивание — 10 баллов

Критерий	Баллы
Конфликт воспроизведён, решён; merge сохраняет обе линии	2
restore применён к известной незаписанной правке; смысл объяснён	2
Ошибочный коммит отменён отдельным revert-коммитом	2
CHANGELOG и теги соответствуют версии, конфигурация корректна	2
Архивы, история, отчёт и проверка восстановления представлены	2

По каждому критерию: 2 — полностью, 1 — частично, 0 — отсутствует или неверно. Для restore в итоговой истории не будет нового коммита, поэтому пояснение в отчёте обязательно.

Помощь при ошибках

Ситуация	Действие
Not a git repository	Проверьте pwd: нужна service-config. Если A1 ещё не выполнен, выполните init, затем настройки.
Author identity unknown	Задайте локальные user.name и user.email после init. Повторите только неудавшийся commit.
Ветка уже существует	Посмотрите git branch и log. Возможно, этап создания уже выполнен. Не создавайте её повторно.
Нет ожидаемого конфликта	Проверьте отдельные коммиты 30 и 120 в разных ветках. Не имитируйте конфликт вручную.
You have unmerged paths	Завершите A5–A6. Пока merge не завершён, не начинайте упражнения Б1–Б3.
LF will be replaced by CRLF	Это предупреждение о переводах строк Windows. Если команда успешна, проверьте status и содержимое; не повторяйте всю работу.
После ввода появился >	Вероятно, незакрыта кавычка. Ctrl+C, затем введите корректную команду целиком.
Revert конфликтует или выбран не тот HEAD	Остановитесь и покажите log преподавателю. Не повторяйте отмену наугад.
Тег v0.2.0 уже существует	Посмотрите git show --stat v0.2.0. Возможно, шаг уже выполнен.
restore-check уже существует	Проверьте прежнюю копию либо используйте новое имя папки во всех командах клонирования.

Если нужно отменить незавершённое слияние

Сразу после A4, пока ещё не редактировали конфликтный файл, допустимо git merge --abort. Оно возвращает к состоянию перед попыткой merge; затем нужно повторить A4–A6. Если уже внесены ценные правки, сначала сохраните их отдельно и уточните действие с преподавателем.

При вопросе сообщите шаг, вывод git status, точную команду и полный текст ошибки. Рабочую папку не удаляйте.

Закрепление и справка

Необязательное сравнение двух версий

```
git --no-pager diff v0.1.0 v0.2.0 -- config.txt
```

```
git --no-pager diff --stat v0.1.0 v0.2.0
```

Первый diff пустой: config.txt в начальной и конечной версии содержит одинаковое значение 60. При этом история проделанной работы сохраняется. Второй diff покажет появление CHANGELOG.txt. Одинаковое итоговое содержимое одного файла не означает одинаковую историю.

Проверьте понимание

1. Почему при слиянии выбран вариант 60, которого не было ни в одной из двух изменённых веток?
2. Почему чистое рабочее дерево не доказывает правильность настройки?
3. Что происходит с ошибочным коммитом после revert?
4. Что сможет проверить преподаватель по bundle и чего нет в release.zip?

Официальные материалы

Установка для Windows: <https://git-scm.com/install/windows>

Слияние: <https://git-scm.com/docs/git-merge>

Восстановление файла: <https://git-scm.com/docs/git-restore>

Отменяющий коммит: <https://git-scm.com/docs/git-revert>

Теги: <https://git-scm.com/docs/git-tag>

Архив файлов: <https://git-scm.com/docs/git-archive>

Перенос истории: <https://git-scm.com/docs/git-bundle>

Клонирование: <https://git-scm.com/docs/git-clone>